

Tests & Quizzes

Question Pool: Quiz 05

[Export](#) | [Print](#)

Question 1: Single Correct - 1.0 point

A developer is auditing a SegWit transaction and observes an input with an empty scriptSig. The corresponding scriptPubKey of the output being spent starts with the SegWit version byte 00. According to the slides, how does the node distinguish between a P2WPKH and a P2WSH spend, and what is the immediate internal consequence of this distinction?

- OA. It checks the Flag field in the transaction header; if the value indicates SegWit usage, it expands a virtual script for validation; otherwise, it processes the input using legacy script execution rules
- OB. It checks the number of items in the witness stack; if there are two items, it expands a virtual P2PKH-style script; if there are more items, it performs hash validation on the final witness element
- C. It checks the byte following the version 00 in the scriptPubKey; if it is 14 (20-byte hash), it expands a virtual P2PKH script; if it is 20 (32-byte hash), it validates the witnessScript against its SHA256 hash
- OD. It checks the Marker and Flag fields together; if SegWit is detected, it executes the scriptSig and witness simultaneously; otherwise, it treats the hash as part of the Merkle tree validation process

Answer Key: C

Question 2: Single Correct - 1.0 point

Transaction malleability in legacy Bitcoin was possible because the txid was computed by hashing the fully serialized transaction including the scriptSig. Which specific property of ECDSA signatures made this exploitable, and what did SegWit do structurally to fix it?

- OA. ECDSA nonce reuse allowed signature forgery; SegWit replaced ECDSA with Schnorr signatures
- B. ECDSA signatures have multiple valid DER encodings for the same (r, s) pair such as adding or removing leading zeros; SegWit excluded the witness data from the txid computation by moving signatures to a separate witness field
- OC. ECDSA public keys could be compressed or uncompressed interchangeably; SegWit mandated compressed keys only
- OD. ECDSA signatures could be replayed across transactions; SegWit added a sequence number to the witness field

Answer Key: B

Question 3: Single Correct - 1.0 point

In a P2WSH multisig execution, the witness field contains OP_0, followed by M signatures, followed by the witnessScript as the last item. During OP_CHECKMULTISIG execution, after popping N public keys and M signatures from the stack, there is still one item remaining at the bottom. What is this item, why is it there, and what happens to it?

- OA. It is the witnessScript hash; it remains because SHA256 verification left it; it is compared against the scriptPubKey
- B. It is the integer 0 pushed by OP_0 at the start of the witness; it exists due to a known bug in OP_CHECKMULTISIG which pops one extra stack item

☐ C. It is the Merkle root of the signatures; it exists for audit purposes; it is dropped by OP_DROP after CHECKMULTISIG

☐ D. It is the number of required signatures M; it exists because OP_2 pushed it; it is verified against the actual count of valid signatures

Answer Key: B

Question 4: Single Correct - 1.0 point

The Bitcoin block header contains a Target_Hash field of only 4 bytes. According to the slides, this field adjusts every 2016 blocks. What is the approximate real-world time this represents and what is the adjustment targeting?

☐ A. Approximately one week, targeting a 5-minute average block generation time

☒ B. Approximately two weeks, targeting a 10-minute average block generation time

☐ C. Approximately one month, targeting a 20-minute average block generation time

☐ D. Approximately three weeks, targeting a 12-minute average block generation time

Answer Key: B

Question 5: Single Correct - 1.0 point

Miniscript is described in the slides as analogous to TypeScript in relation to JavaScript. A spending policy states: Alice can spend alone, OR Bob can spend after 30 days, OR Alice and Bob together can spend anytime. When this policy is compiled to Bitcoin Script, which opcodes from the slides are necessarily involved to enforce the time constraint on Bob's spending path?

☐ A. OP_CHECKLOCKTIMEVERIFY and OP_IF

☒ B. OP_CHECKSEQUENCEVERIFY and OP_IF with OP_ELSE and OP_ENDIF

☐ C. OP_CHECKLOCKTIMEVERIFY with OP_CHECKSIG only

☐ D. OP_CHECKSEQUENCEVERIFY with OP_CHECKMULTISIG only

Answer Key: B

Question 6: Single Correct - 1.0 point

A vendor receives payment and wants to verify that a specific transaction is included in Block 720,518 without downloading all transactions in that block. Using a Merkle proof, what is the computational complexity of this verification compared to naive verification of all transactions?

☐ A. $O(n)$ for Merkle proof vs $O(n^2)$ for naive

☒ B. $O(\log n)$ for Merkle proof vs $O(n)$ for naive

☐ C. $O(1)$ for Merkle proof vs $O(\log n)$ for naive

☐ D. $O(\log n)$ for Merkle proof vs $O(\log n)$ for naive, but with smaller constants

Answer Key: B

Question 7: Single Correct - 1.0 point

In a P2WPKH transaction, the scriptPubKey stored in the output is 00 14 <20-byte-hash>. When this output is being spent, Bitcoin internally constructs a virtual script to execute. What exactly is this virtual script and where does the 20-byte value come from?

☐ A. OP_HASH256 <20-byte-hash> OP_EQUALVERIFY OP_CHECKSIG, where the 20-byte value is derived by applying SHA256 twice on the public key provided in the witness

- B. OP_DUP OP_HASH160 <20-byte-pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG, where the 20-byte value is taken directly from the scriptPubKey of the previous transaction output
- C. OP_CHECKSIG OP_HASH160 <20-byte-hash> OP_EQUALVERIFY, where the 20-byte value is reconstructed from the witness data and validated against the transaction input
- D. OP_SHA256 <20-byte-pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG, where the 20-byte value is expanded from the witness field and used for verification

Answer Key: B

Question 8: Single Correct - 1.0 point

In a P2WSH multisig transaction, the scriptPubKey of the output being spent begins with 00 20 followed by 32 bytes. Before executing the witness script, what validation step must succeed, and what specifically is being compared against those 32 bytes?

- A. The SHA256 hash of the concatenated public keys must match the 32 bytes
- B. The SHA256 hash of the witnessScript provided in the witness field must equal the 32-byte value in the scriptPubKey of the previous transaction output
- C. The HASH160 of the witnessScript must match the 32 bytes, confirming script integrity
- D. The double SHA256 of all witness items concatenated must equal the 32 bytes stored in scriptPubKey

Answer Key: B

Question 9: Single Correct - 1.0 point

According to the slides, an OP_RETURN output has a value of 0 BTC and is explicitly unspendable. What is the practical implication of this for the UTXO database maintained by Bitcoin nodes?

- A. OP_RETURN outputs are stored permanently in the UTXO database since they can never be spent and removed
- B. OP_RETURN outputs are never added to the UTXO database since they are provably unspendable, meaning they do not bloat the chainstate
- C. OP_RETURN outputs are stored temporarily in the UTXO database until the next block confirmation
- D. OP_RETURN outputs cause the entire transaction to be rejected as invalid by full nodes

Answer Key: B

Question 10: Single Correct - 1.0 point

A developer wants to lock funds such that they can only be spent after a specific block height has been reached AND requires two out of three signatories to approve. According to the Bitcoin script opcodes covered in the slides, which combination of opcodes would be needed to enforce both conditions simultaneously?

- A. OP_CHECKLOCKTIMEVERIFY combined with OP_CHECKMULTISIG
- B. OP_CHECKSEQUENCEVERIFY combined with OP_CHECKSIG
- C. OP_CHECKLOCKTIMEVERIFY combined with OP_CHECKSIG
- D. OP_CHECKSEQUENCEVERIFY combined with OP_CHECKMULTISIG

Answer Key: A

Return